

GABRIEL POPA

Senior Software Engineer - React • Go • AWS

Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | <https://www.linkedin.com/in/gabriel-popa-12890b414/>

ABOUT ME

Senior Software Engineer with **10+ years** of experience building business platforms, API-driven products, and data-heavy workflows across the **software industry**, with strongest expertise in **Go**, **React**, distributed backend services, and modern web application delivery. Built reliable systems that balance performance, maintainability, and product speed, while handling feature ownership, production debugging, legacy cleanup, and architecture improvements across multiple environments. Core hands-on range includes **Go 1.17–1.23**, **React 16–18**, **TypeScript 4.x–5.x**, **PostgreSQL**, **Redis**, **Docker**, and **AWS**, with proven success improving slow endpoints, stabilizing brittle integrations, and turning evolving requirements into clean, supportable software used in daily operations.

SKILLS

- **Frontend: React 16–18, TypeScript 4.x–5.x**, JavaScript ES6+, Redux Toolkit, React Query 3.x–5.x, HTML5, CSS3, SCSS, responsive UI design, reusable component architecture, form validation, accessibility-aware development
- **Backend: Go 1.17–1.23**, Gin 1.x, Echo 4.x, Fiber 2.x, REST API design, gRPC basics, microservices, authentication, authorization, background jobs, webhook processing, integration services, legacy service refactoring
- **Data & Messaging: PostgreSQL 12–16**, MySQL 5.7–8.0, **Redis 5–7**, SQL, database tuning, query optimization, caching strategies, RabbitMQ 3.x, Kafka basics, asynchronous processing, reporting workflows
- **Cloud & DevOps: AWS, Docker 20.x–26.x**, Kubernetes 1.24–1.30 basics, ECS, EC2, S3, RDS, CloudWatch, CI/CD pipelines, GitHub Actions, GitLab CI, Linux, Nginx
- **Observability / Quality / Security:** structured logging, OpenTelemetry basics, Prometheus basics, Grafana basics, unit testing, integration testing, API testing, **Jest 27–29**, **React Testing Library 13–16**, Go testing, RBAC, JWT, rate limiting, error monitoring, secure API practices

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania (*Jan 2024 – May 2026*)

- Led delivery of internal and customer-facing platform modules using **Go 1.21–1.22**, **React 18**, and **TypeScript 5.x**, translating fragmented operational requirements into stable workflows for dashboards, approvals, reporting, and administration across fast-changing product areas.
- Modernized older backend services into cleaner **Go 1.21–1.22** service boundaries with improved **handler**, **service**, and **repository** separation, which reduced regression risk and made new feature delivery noticeably safer for the team.
- Built reusable **React 18** components for **data tables**, **filters**, **modal flows**, and **validation-heavy forms**, cutting duplicated frontend effort and improving implementation consistency across newly introduced product modules by **27%**.
- Solved a recurring **production timeout** affecting large reporting requests by profiling slow **SQL** paths, tightening payload size, and introducing **pagination** with smarter query filters, which improved response stability by **34%**.
- Implemented a **permission-aware administration area** with **JWT-secured APIs** and **role-based UI rendering**, reducing unauthorized action attempts and manual intervention work for operations staff by **22%**.
- Fixed an intermittent **data mismatch bug** caused by stale **Redis** cache reads after rapid record updates by refining invalidation timing and clarifying API refresh behavior in the **React** client.
- Introduced **structured logging** and trace-friendly **request correlation** around critical API paths, which shortened root-cause analysis time for multi-step failures and helped the team resolve production issues about **31%** faster.
- Delivered new **asynchronous processing** for heavy export and notification tasks using background workers in **Go**, replacing request-blocking behavior that had previously caused browser failures and support complaints during peak usage.

- Refined **API error contracts** and frontend fallback handling after inconsistent validation responses created confusing user behavior, which improved form completion reliability and reduced repeat submission issues by **19%**.
- Supported migration of brittle integration logic into more maintainable **service adapters**, allowing third-party connection changes to be released with lower risk and fewer hotfixes after deployment.
- Mentored developers on practical backend debugging, **code review** standards, and safer rollout patterns, helping the team ship larger changes with stronger confidence and fewer avoidable production defects.
- Partnered with **product** and **QA** teams to break down ambiguous requirements into incremental deliveries, which improved release predictability and reduced late-sprint rework across cross-functional initiatives by **18%**.

Software Engineer

Next Apps | Ghent, Belgium (Nov2020 – Nov 2023)

- Built full-stack product capabilities using **Go 1.18–1.20**, **React 17–18**, and **PostgreSQL**, contributing to customer workflows, admin tooling, and integration-heavy screens where backend correctness and responsive UI behavior were equally important.
- Migrated selected frontend areas from older **React** patterns into a more maintainable **React 17–18** component and **hooks** approach, improving state clarity and reducing duplicated logic across frequently modified screens.
- Implemented **API endpoints** in **Go 1.18–1.20** for **search**, **filtering**, **bulk actions**, and **status-driven workflows**, giving frontend modules cleaner contracts and reducing edge-case handling complexity in the browser.
- Investigated a difficult issue where **concurrent updates** occasionally overwrote recent changes, then solved it by tightening **transaction** behavior and clarifying optimistic update handling, which reduced record conflict incidents by **24%**.
- Improved slower **list** and **reporting pages** by rewriting costly **SQL joins**, adding targeted indexes, and removing unnecessary response fields, which made large datasets easier to use and improved load speed by **29%**.
- Built reusable **form** and **table abstractions** in **React** that supported **validation**, **conditional rendering**, and **permission-sensitive controls**, which accelerated new screen delivery and reduced UI defects by **21%**.
- Stabilized third-party **webhook ingestion** after malformed payloads caused partial processing failures by adding stricter validation, retry-safe logic, and clearer error logs for operations review.
- Introduced **Redis** caching for hot read endpoints with explicit invalidation rules, lowering repeated database pressure while keeping business-critical screens accurate enough for real operational use.
- Helped migrate legacy service behavior from mixed **controller logic** into clearer backend layers, making **test coverage** more practical and reducing the risk of accidental breakage during product expansion.
- Worked closely with **QA** to reproduce production-only bugs involving browser timing and asynchronous requests, then added safer loading and retry behavior that reduced flaky user-facing failures by **17%**.
- Contributed to **CI/CD** refinement and release verification practices, which improved deployment consistency and reduced rollback frequency during periods of active feature delivery and integration change.

Software Developer

Datamart | Stockholm, Sweden (Feb 2016 – Sep 2020)

- Developed backend-heavy business applications using **Go 1.12–1.16**, JavaScript, **SQL**, and server-rendered web patterns, supporting reporting, internal workflows, and data processing features for operational teams.
- Helped introduce **Go 1.12–1.16** services into an environment still carrying older application patterns, giving the team a practical path toward faster APIs and simpler background job implementations.
- Built and maintained **REST-style endpoints** consumed by frontend modules and internal tools, improving data access consistency and reducing duplicated query logic spread across legacy application areas.
- Solved a recurring **export failure** where larger datasets exhausted memory during generation by replacing all-at-once processing with **streamed output handling**, improving reliability for reporting users by **26%**.
- Improved slow **data-heavy screens** by reviewing execution plans, simplifying joins, and adding missing indexes in **PostgreSQL** and **MySQL**, which made operational dashboards significantly more responsive.
- Implemented early **React 16** components for selected interactive pages while keeping compatibility with existing frontend structure, allowing the product to modernize gradually without risky large-scale rewrites.
- Fixed a frustrating issue where **background jobs** occasionally processed duplicate records after retries by adding **idempotency checks** and clearer **job-state tracking** in backend processing logic.
- Built reusable utilities for **validation**, **formatting**, and **API response shaping**, which reduced repetitive code and made new backend endpoints easier to implement and support over time.

- Supported integration work with external services and internal batch flows, handling **payload inconsistencies** and edge cases that previously caused support teams to rely on manual correction steps.
- Worked under senior guidance on **production troubleshooting**, learning to reproduce issues from logs and user behavior reports, then delivering focused fixes that improved day-to-day platform stability by **14%**.

Junior Software Developer

Berg Software | Timișoara, Romania (Sep2014 – Mar 2016)

- Supported web application development with **Go 1.3–1.5**, JavaScript, **SQL**, HTML, and CSS, contributing to smaller backend features, bug fixes, admin pages, and maintenance tasks within a structured team environment.
- Assisted with early **Go 1.3–1.5** service work for lightweight internal endpoints and utility processes, gaining hands-on experience with **request handling**, **JSON responses**, and basic service organization.
- Built small **UI improvements** and interactive page elements, helping older screens behave more clearly for users while keeping implementation aligned with the frontend conventions used by the wider team.
- Fixed a recurring **validation mismatch** between browser checks and server rules that had caused confusing submission failures, which reduced rejected form attempts and support noise by **16%**.
- Investigated an intermittent **login issue** related to **session timing** and **cookie behavior**, then helped apply a safer configuration that made authentication flow more predictable for everyday users.
- Contributed to database updates and **correction scripts** when problematic legacy records triggered application errors, supporting smoother releases by removing edge cases before they reached production.
- Helped extract duplicated backend **helpers** into reusable functions, making repeated maintenance work simpler and reducing the chance of inconsistent fixes across related modules.
- Participated in debugging **production issues** by tracing logs, reproducing bugs locally, and applying carefully scoped patches, building practical habits around safe changes and reliable testing.
- Learned to adapt across **backend**, **frontend**, and **data** tasks as project needs shifted, which made it easier for the team to cover urgent work without slowing delivery on active customer requests.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (2010 – 2014)

- Academic foundation covered software engineering, algorithms, database systems, and practical application development, providing the base for long-term specialization in backend engineering and scalable service design.

CERTIFICATIONS

Go: Building Microservices — 2024

- Demonstrated practical capability in designing **service boundaries**, handling **inter-service communication**, and building maintainable **backend systems** with Go.

Docker & Kubernetes Foundations — 2023

- Validated working knowledge of **containerized deployment**, **runtime isolation**, and basic **orchestration patterns** commonly used for modern backend service delivery.

PostgreSQL for Developers — 2022

- Demonstrated hands-on understanding of **relational modeling**, **query optimization**, **indexing**, and **performance-aware backend data access patterns**.